

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto: GS Stock & Facturación.

Link del documento de software:

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Link al documento:

Link de la presentación:

https://www.canva.com/design/DAGuJZ_oZxw/jETyDEjXnMS_3VXiR0IltQ/edit?utm_content=DAGuJZ_oZxw&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

Product Backlog

Lista de tareas

GSS-90 Tarea principal: Terminar implementación de WebSockets

- **GSS-91** Emitir eventos en cambios de inventario
- **GSS-92** Escuchar eventos y actualizar UI
- **GSS-93** Lógica para descontar stock al crear pedido en la db con websocket

GSS-94 Interfaz gestión clientes

- **GSS-95** Verificar modelo en la base de datos cliente: nombre, deudas, dirección y teléfono
- **GSS-96** Endpoint CRUD para clientes
- **GSS-97** Formulario de edición/creación/eliminar
- **GSS-98** Componente Tabla clientes con campos: nombre, dirección, teléfono, deuda, historial de pedidos

GSS-99 Tarea principal: Facturación

- **GSS-100** Modificar formulario de venta con los campos: nombre, dirección, teléfono, NIT, productos y pago total
- **GSS-101** Enpoints Post para crear, guardar y generar facturas
- **GSS-102** Enpoints GET para autollenado de campos: nombre, dirección, teléfono, NIT
- **GSS-103** Conexión de los endpoint con la UI

GSS-105 Historial de ventas

- **GSS-106** Agregar apartado 'Historial facturas' en ventas
- **GSS-107** Componente con lista de todas las facturas generadas usando el método GET facturas
- **GSS-108** Filtros por fecha y nombre de cliente
- **GSS-109** Endpoint get para obtener historial de todas las facturas generadas

GSS-110 Tarea principal Test

- **GSS-111** Test agregar producto
- **GSS-112** Test CRUD usuarios
- **GSS-113** Test agregar producto a inventario
- **GSS-114** Test Modales
- **GSS-115** Test componente UserTable
- **GSS-116** Test componente user view
- **GSS-117** Arreglos Product Owner
- **GSS-118** Modificar entrada al campo "codigo zapato" a alfanumérico en UI inventario
-

Pila del Sprint

ID	Nombre de la tarea	Puntos de Historia
GSS-90	Tarea principal: Terminar implementación de WebSockets	3
GSS-91	Emitir eventos en cambios de inventario	1
GSS-92	Escuchar eventos y actualizar UI	1
GSS-93	Lógica para descontar stock al crear pedido en la db con websocket	1
GSS-94	Interfaz gestión clientes	3
GSS-95	Verificar modelo en la base de datos cliente: nombre, deudas, dirección y teléfono	1
GSS-96	Endpoint CRUD para clientes	1
GSS-97	Formulario de edición/creación/eliminar	1
GSS-98	Componente Tabla clientes con campos: nombre, dirección, teléfono, deuda, historial de pedidos	1
GSS-99	Tarea principal: Facturación	3
GSS-100	Modificar formulario de venta con los campos: nombre, dirección, teléfono, NIT, productos y pago total	1
GSS-101	Endpoints POST para crear, guardar y generar facturas	1
GSS-102	Endpoints GET para autollenado de campos: nombre, dirección, teléfono, NIT	1

GSS-103	Conexión de los endpoint con la UI	1
GSS-105	Historial de ventas	3
GSS-106	Agregar apartado ‘Historial facturas’ en ventas	1
GSS-107	Componente con lista de todas las facturas generadas usando el método GET facturas	1
GSS-108	Filtros por fecha y nombre de cliente	1
GSS-109	Endpoint GET para obtener historial de todas las facturas generadas	1
GSS-110	Tarea principal Test	3
GSS-111	Test agregar producto	1
GSS-112	Test CRUD usuarios	1
GSS-113	Test agregar producto a inventario	1
GSS-114	Test Modales	1
GSS-115	Test componente UserTable	1
GSS-116	Test componente UserView	1
GSS-117	Arreglos Product Owner	2
GSS-118	Modificar entrada al campo “código zapato” a alfanumérico en UI inventario	1

Calendario de planificación

Fecha	Tarea
01/07/2025	GSS-90 Terminar implementación de WebSockets
02/07/2025	GSS-91 Emitir eventos en cambios de inventario
03/07/2025	GSS-92 Escuchar eventos y actualizar UI
04/07/2025	GSS-93 Lógica para descontar stock al crear pedido en la db con websocket
05/07/2025	GSS-94 Interfaz gestión clientes
06/07/2025	GSS-95 Verificar modelo en base de datos cliente
07/07/2025	GSS-96 Endpoint CRUD para clientes
08/07/2025	GSS-97 Formulario de edición/creación/eliminar
09/07/2025	GSS-98 Componente tabla clientes
10/07/2025	GSS-99 Tarea principal: Facturación
11/07/2025	GSS-100 Modificar formulario de venta
12/07/2025	GSS-101 Endpoints POST para facturas
13/07/2025	GSS-102 Endpoints GET para autollenado
14/07/2025	GSS-103 Conexión endpoints con UI
15/07/2025	GSS-105 Historial de ventas
16/07/2025	GSS-106 Agregar apartado 'Historial facturas'
17/07/2025	GSS-107 Lista de facturas generadas (GET facturas)
18/07/2025	GSS-108 Filtros por fecha y nombre de cliente
19/07/2025	GSS-109 Endpoint GET para historial de facturas
20/07/2025	GSS-110 Tarea principal de pruebas
21/07/2025	GSS-111 Test agregar producto
22/07/2025	GSS-112 Test CRUD usuarios
23/07/2025	GSS-113 Test agregar producto a inventario

24/07/2025	GSS-114 Test modales
24/07/2025	GSS-115 Test componente UserTable
25/07/2025	GSS-116 Test componente UserView
25/07/2025	GSS-117 Arreglos Product Owner
25/07/2025	GSS-118 Modificar entrada al campo “código zapato” en UI inventario

Incremento

Link al repositorio: <https://github.com/JosFer720/GS-Stock-y-Facturacion-Frontend>

Estimación de Tiempo y Costo del desarrollo

Metodo Function Points

1. Cálculo (PCU)

$$PCU = FPA + FPCU$$

$$PCU = 5 + 85 = 90$$

2. Cálculo (FPA)

Peso de actores	Complejidad
Jefe	3
Secretaria	1
Vendedor	1
Total	5

3. Cálculo (FPCU)

Peso caso de usos	Peso
Gestión de inventario	15
Facturación	15
Ventas	15
Autenticación	10
Gestion clientes	10
Reportes	10
Consultas básicas	5
Alertas de stock	5

$$FCPU = (15 \times 3) + (10 \times 3) + (5 \times 2) = 85$$

4. Calculo (PCUA)

$$PCUA = PCU \times FCT \times FA$$

$$PCUA = 90 \times 0.89 \times 0.74 = 59$$

5. Cálculo (FCT)

Factor	Valoración
Sistema distribuido (Docker, microservicios)	4
Rendimiento (gestión de inventario en tiempo real)	4
Seguridad (JWT, roles)	5
Transacción de ventas.	4
Entrenamiento a usuarios	2
Reusabilidad de código	3
Portabilidad	3

Transacciones ACID (ventas, inventario)	4
---	---

$$FCT = 0.6 + (0.01 \times \sum 29) = 0.89$$

6. Cálculo (FA)

Factor	Valoración
Familiaridad con el modelo de proyecto	4
Experiencia en orientación a objetos	5
Capacidad del lider	4
Motivación	4
Estabilidad de requerimientos	3
Dificultad del lenguaje	2
	22

$$FA = 1.4 + (-0.03 \times 22) = 0.74$$

7. Cálculo (E)

$$E = PCUA \times FC$$

$$E = 59 \times 20 = 1180h$$

8. Calculo costo estimado

$$\text{Costo total} = 1180h \times \$20 = \$23\,600 = Q181\,194$$

Método utilizado

Para estimar el esfuerzo de desarrollo, utilizamos la técnica de Puntos de Casos de Uso (PCU), ya que nos pareció una forma objetiva y estructurada de calcular el trabajo necesario. Primero identificamos y clasificamos a los actores del sistema para calcular el Factor de Peso de Actores (FPA), y luego analizamos la complejidad de cada caso de uso para determinar el Factor de Peso de Casos de Uso (FPCU).

Sumando ambos, obtuvimos los PCU brutos. Después, ajustamos esa cifra teniendo en cuenta factores técnicos como requisitos de rendimiento o seguridad (FCT), así como factores del entorno como la experiencia del equipo o qué tan estables eran los requerimientos (FA). Finalmente, multiplicamos los Puntos de Casos de Uso Ajustados (PCUA) por una productividad estándar, en nuestro caso, usamos 20 horas por PCUA para determinar el esfuerzo total.

Costo de tiempo y desarrollo

El resultado que obtuvimos fue un estimado de 1,180 horas de trabajo, lo que se traduce en unos 7.4 meses de desarrollo si asumimos una jornada de 160 horas al mes. En cuanto al costo, calculamos que serían unos \$23,600 dólares, considerando una tarifa promedio de \$20 por hora. Convertido a moneda local, eso representa aproximadamente Q181,194, usando un tipo de cambio de 7.68 quetzales por dólar.

Este cálculo incluye el desarrollo base, más los ajustes por complejidad técnica y factores ambientales, aunque no contempla posibles gastos imprevistos

Prueba

Backend

En el desarrollo del backend utilizamos Jest y Supertest, ya que permiten validar de manera eficiente el funcionamiento de las rutas, controladores y servicios internos. Estas tecnologías fueron elegidas porque se integran naturalmente con aplicaciones en Express, ofrecen simplicidad en su configuración y permiten realizar pruebas automatizadas.

Jest cuenta con una capacidad que permite probar módulos, controladores y rutas de forma aislada, detectando errores antes de pasar a producción. Además, es compatible con pruebas síncronas y asíncronas, lo que resulta ideal para operaciones que interactúan con bases de datos. Sus funcionalidades como mocks y spies permiten simular el comportamiento de dependencias externas y aislar las pruebas, evitando interferencias de otros módulos.

Por otro lado, utilizamos Supertest como biblioteca complementaria enfocada en pruebas de integración de las APIs HTTP. Esta herramienta permite simular solicitudes HTTP directamente hacia las rutas definidas en Express, sin necesidad de levantar un servidor real. De esta manera, es posible validar las respuestas, códigos de estado, encabezados y el contenido devuelto por cada endpoint. Supertest es especialmente útil para comprobar el correcto funcionamiento del backend como un todo.

Frontend

En el desarrollo del frontend, se utilizó Vitest como herramienta principal para la creación de pruebas unitarias, en conjunto con @vue/test-utils, que permite montar y simular interacciones con componentes Vue de forma aislada. Ambas herramientas fueron seleccionadas por su excelente integración con el entorno de desarrollo en Vue y Vite, el cual ya era utilizado por el equipo en el proyecto. Vitest ofrece una experiencia similar a Jest pero optimizada para entornos Vite, con ejecución rápida, soporte para pruebas con JSDOM. Esto permitió escribir pruebas sin necesidad de configurar entornos complejos.

Por otro lado, @vue/test-utils es el paquete oficial para testing de componentes Vue. Permite montar componentes, pasarles props, simular eventos y verificar su estado y estructura DOM, lo cual es fundamental para validar la lógica visual de la aplicación sin necesidad de un navegador real.

Backend

Prueba de agregar productos:

Tiene como objetivo verificar el funcionamiento del endpoint que permite agregar nuevos productos. Primero, se genera un token válido que simula un usuario autenticado, necesario para autorizar las peticiones. Luego, la prueba valida tres escenarios. El primero verifica que el sistema permite agregar un producto correctamente cuando se envían todos los datos requeridos y el token de autenticación, probando que la respuesta sea exitosa y tenga la información esperada del producto. El segundo comprueba que la solicitud sea rechazada si no se envía el token de autenticación, asegurando así que el acceso esté protegido. La tercera validación es enviar datos incompletos, el sistema debe responder con un error indicando que faltan campos requeridos.

Por qué es importante: Permite asegurar que la funcionalidad crítica de agregar productos esté protegida, validada y que solo usuarios autenticados puedan modificar el inventario con datos completos.

Video: <https://youtu.be/2buOIOdQPis>

Prueba CRUD usuarios:

Este test valida que el componente UsersTable funcione correctamente en cuanto a su renderizado e interacciones, asegurando que la tabla muestre adecuadamente la lista de usuarios con todos sus datos (ID, Nombre, Apellido, Email, ID Rol, Estado) y aplique estilos condicionales según el estado del usuario. También cubre casos como el manejo de listas vacías mostrando mensajes adecuados, y verifica que la selección de usuarios mediante clics en filas funcione correctamente, manteniendo la integridad de los datos.

Por qué es importante: Garantiza que la administración de usuarios se realice de forma visualmente clara, funcional y sin errores, lo que mejora la usabilidad y fiabilidad del sistema.

Video: <https://youtu.be/CFgEQb19IVw>

Prueba de agregar inventario:

Esta prueba unitaria verifica el correcto funcionamiento de las rutas de inventario definidas en inventory.js, específicamente las operaciones de agregar y actualizar productos. Las pruebas incluyen casos como agregar un producto nuevo o actualizar el stock.

Por qué es importante: Asegura que los movimientos de inventario (alta y actualización) se ejecuten correctamente, manteniendo la consistencia y precisión de los datos del sistema.

Video: <https://www.youtube.com/watch?v=FX00dnPLHHA>

Frontend

Prueba ModalMessage:

Se realizó una prueba unitaria al componente ModalMessage.vue, el cual muestra un modal con un mensaje y botones para cerrar o confirmar. La prueba incluyó tres validaciones: que el mensaje, título y clase CSS del modal aparezcan correctamente al activar la prop show; que el componente no se renderice cuando show es igual a false; y que al hacer clic en el botón “Aceptar” se emita el evento close.

Por qué es importante: Verifica que los mensajes modales se muestren y oculten correctamente, y que la interacción del usuario con ellos sea fluida y sin errores, lo que mejora la experiencia de usuario.

Video: <https://youtu.be/w1QydrFBizg>

Prueba componente UsersTable:

Las pruebas del archivo usuarios.test.js comprueban todas las operaciones CRUD de la API REST de usuarios, validando la creación, consulta, actualización, desactivación y eliminación de usuarios, además del filtrado por estado. Se verifica que todos los endpoints estén protegidos por autenticación JWT y que las peticiones no autorizadas sean rechazadas, así como que se manejen correctamente los errores al recibir datos inválidos.

Por qué es importante: Protege la lógica del backend, asegurando que los usuarios sean manipulados de forma segura, validada y controlada, reforzando la seguridad de la aplicación.

Video: <https://youtu.be/tsCmj2Kye3g>

Prueba componente LoginView:

Las pruebas del archivo LoginView.spec.js comprueban la funcionalidad completa del componente de inicio de sesión, validando el renderizado correcto de los elementos de

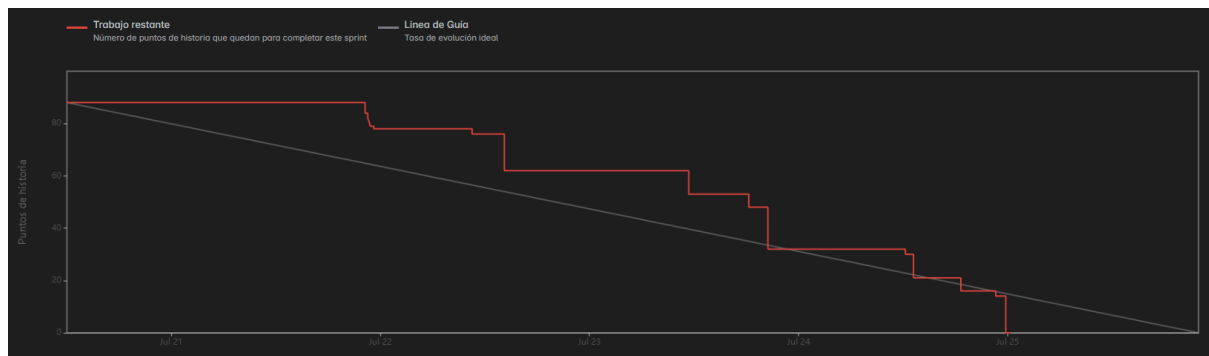
la interfaz (formulario, campos de entrada, botones y enlaces), la actualización de datos del formulario, la validación de campos obligatorios, y el manejo de respuestas del servidor.

Por qué es importante: Garantiza que el proceso de autenticación funcione correctamente desde el frontend, brindando acceso seguro a los usuarios válidos y evitando errores en el inicio de sesión.

Video: https://www.youtube.com/watch?v=UF_Sg2sHZKI

Resultados del Sprint

Gráfico Burndown

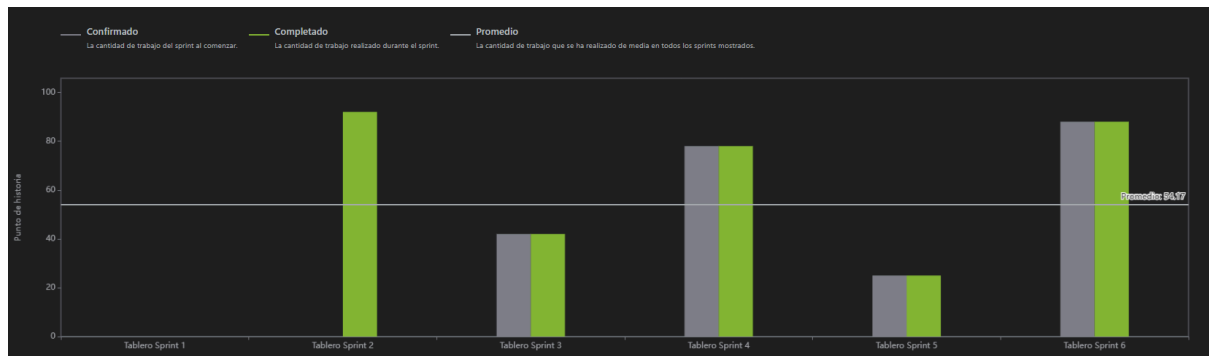


En este sprint trabajamos principalmente en la generación de facturas y el CRUD de clientes. Al inicio nos costó un poco arrancar, como se ve en el gráfico, ya que durante el primer día no hubo reducción en los puntos de historia. Pero una vez nos organizamos y definimos bien el flujo de trabajo, empezamos a avanzar de forma más constante. A partir de ahí, cada entrega y tarea cerrada se reflejó claramente en el burndown, con una disminución progresiva en el trabajo restante.

Me parece importante resaltar que logramos completar todas las tareas antes de la fecha final del sprint, lo cual demuestra que supimos mantener un buen ritmo sin saturarnos.

Personalmente sentí que el equipo trabajó muy coordinado y enfocado, lo que nos permitió mantenernos cerca de la línea de guía ideal.

Métrica de velocidad



A lo largo de los últimos sprints hemos ido consolidando una buena capacidad de entrega. Como se puede ver en el gráfico, nuestra velocidad promedio ha sido de 54,17 puntos de historia, y en general hemos mantenido un rendimiento bastante estable, especialmente en los sprints 3, 4 y 6, donde lo confirmado y lo completado estuvieron al mismo nivel. Me parece que esto refleja una mejora clara en nuestras estimaciones y en la forma en que gestionamos el trabajo dentro del equipo.

Indicador numérico del éxito del sprint

Puntuación: 9/10

En el Sprint 5, el equipo logró completar las 27 tareas planificadas, alcanzando un 100% de cumplimiento. Se implementaron correctamente módulos críticos como la gestión de clientes, la facturación y las pruebas tanto en frontend como backend. Destaca la implementación de WebSockets para actualizar en tiempo real la UI e inventario tras registrar ventas, así como la automatización en la generación de facturas. También se finalizó el historial de facturación con filtros funcionales y la conexión adecuada de endpoints con la interfaz. El equipo mantuvo un ritmo de trabajo constante tras una fase inicial lenta, cumpliendo todos los objetivos sin retrasos.

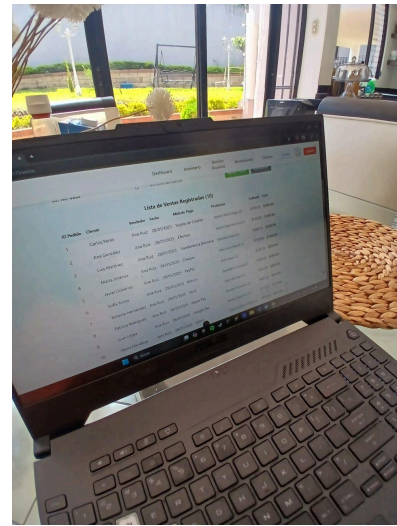
El único motivo por el que no se califica con un 10 es que al inicio del sprint hubo una falta de avance visible, lo que afectó temporalmente la proyección en el burndown. Sin embargo, esta situación fue corregida rápidamente gracias a una buena coordinación interna. El equipo mostró una clara mejora en la estimación de esfuerzo y planificación de tareas, lo que se reflejó en una ejecución estable y en un incremento funcional y bien recibido por el Product Owner.

Discusión del éxito del sprint

El Sprint 5 fue valorado con 9/10 debido al excelente desempeño del equipo y la entrega completa de funcionalidades. Se abordaron correctamente módulos clave como el CRUD de clientes y el sistema de facturación, integrando tanto lógica de negocio como visual, además de asegurar el comportamiento esperado a través de pruebas unitarias e integración. Las tareas se completaron según lo planeado, y el incremento presentado al Product Owner fue funcional y visualmente satisfactorio.

A pesar de un arranque lento, el equipo se reorganizó y logró un progreso sostenido, tal como se evidencia en la gráfica burndown. La motivación y coordinación permitieron cerrar todas las tareas antes de la fecha límite. El feedback recibido durante la revisión fue positivo, destacando la emoción del Product Owner al ver las facturas generadas y el sistema de ventas funcional. Las observaciones se centraron en mejoras de UI, lo cual representa una oportunidad de refinamiento más que una falla estructural del sprint.

Evidencia de muestra del incremento desarrollado a usuarios finales



En este sprint le presentamos al Product Owner los avances en la pantalla de gestión de clientes, donde le mostramos cómo registrar un cliente con todos sus datos, así como las mejoras en la pantalla de ventas. Ahora, al realizar una venta, el sistema descuenta automáticamente la cantidad vendida del inventario y genera una factura que se descarga lista para su envío. El Product Owner se mostró bastante emocionado al ver las facturas con el nombre de la empresa y nos expresó lo contento que estaba con el funcionamiento en general, especialmente con la actualización automática del inventario en tiempo real.

Durante la revisión, nos indicó algunos cambios importantes a realizar: mejorar la UI del inventario para que al agregar un producto se puedan ingresar cantidad, tallas, ID del zapato, precio y estado; ajustar la tabla de inventario para mostrar cantidad por tallas y precios; y modificar la UI de ventas para calcular totales y hacer la resta de inventario de forma precisa. A pesar de estas observaciones, nos felicitó por los avances y se mostró muy satisfecho con el progreso del proyecto.

Retrospectiva del sprint

Durante este sprint, el equipo demostró una ejecución sólida y bien organizada. Se completaron las 27 tareas planificadas, incluyendo funcionalidades clave como la gestión de clientes, el módulo de facturación y las pruebas en ambos entornos (frontend y backend). La implementación de WebSockets permitió una interacción más fluida con la interfaz, especialmente en la actualización automática del inventario. Además, se logró una presentación exitosa del incremento al Product Owner, quien quedó satisfecho con el avance general. La colaboración entre los integrantes fue constante y eficiente, manteniéndose siempre cerca de la línea de trabajo ideal del burndown.

Sin embargo, el inicio del sprint fue lento, con un primer día sin avance reflejado en la gráfica. Aunque se logró recuperar el ritmo rápidamente, esto evidenció la necesidad de una mejor organización desde el primer día. También se identificaron oportunidades de mejora en la interfaz, ya que el Product Owner sugirió varios ajustes visuales y funcionales. Para próximos sprints, el equipo acordó dedicar más tiempo a validar los flujos de UI y mantener un ritmo sostenido desde el comienzo para asegurar entregas fluidas y sin contratiempos.

Sistema de control:

- [Link al jira](#)

Excel con control de tiempos

- [Horario software.xlsx](#)

Anexo

Evidencia de ejecución de pruebas unitarias

Backend:

Prueba de agregar productos:

```
FINAL
ruiz@MacBook-Air-206 GS-Stock % docker exec -it gs-stock-backend-1 n
ckend@1.0.0 test
st

$ tests/product.test.js
$T /api/productos
✓ debería agregar un producto correctamente (41 ms)
✓ debería rechazar si falta el token (3 ms)
✓ debería rechazar si faltan campos requeridos (2 ms)

Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
Snapshots: 0 total
Time: 0.275 s, estimated 1 s
Ran all test suites.
```

Prueba CRUD usuarios:

```
> backend@1.0.0 test
> jest usuarios.test.js

PASS tests/usuarios.test.js
  CRUD de Usuarios
    ✓ debería crear un nuevo usuario (80 ms)
    ✓ debería obtener un usuario por ID (7 ms)

Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 1.002 s
Ran all test suites matching /usuarios.test.js/i.
Jest did not exit one second after the test run has completed.
```

Prueba de agregar inventario:


```
> jest

PASS tests/inventory.test.js
PASS tests/usuarios.test.js
PASS tests/product.test.js

Test Suites: 3 passed, 3 total
Tests: 15 passed, 15 total
Snapshots: 0 total
Time: 1.029 s
Ran all test suites.
```

Frontend:

Prueba ModalMessage:

```
✓ src/components/__tests__/HelloWorld.spec.js (1 test) 20ms
✓ src/tests/unit/ModalMessage.spec.ts (3 tests) 35ms

Test Files 2 passed (2)
Tests 4 passed (4)
Start at 05:08:34
Duration 2.60s (transform 1.85s, setup 0ms, collect 1.86s,
PASS Waiting for file changes...
press h to show help, press q to quit
```

Prueba componente UserTable:

```
✓ src/components/__tests__/HelloWorld.spec.js
✓ src/tests/unit/ModalMessage.spec.ts (3 tests)
✓ src/tests/unit/UsersTable.spec.js (8 tests)

Test Files 3 passed (3)
Tests 12 passed (12)
Start at 21:32:50
Duration 1.63s (transform 424ms, setup 0ms)
PASS Waiting for file changes...
```

Prueba componente LoginView:

```
/ src/tests/unit/ModalMessage.spec.ts (3 tests)
/ src/tests/unit/UsersTable.spec.js (8 tests) 1
/ src/tests/unit/LoginView.spec.js (13 tests) 2

est Files  4 passed (4)
  Tests  25 passed (25)
  Start at 02:32:11
  Duration 1.96s (transform 515ms, setup 0ms,

PASS Waiting for file changes...
press h to show help, press q to quit
```